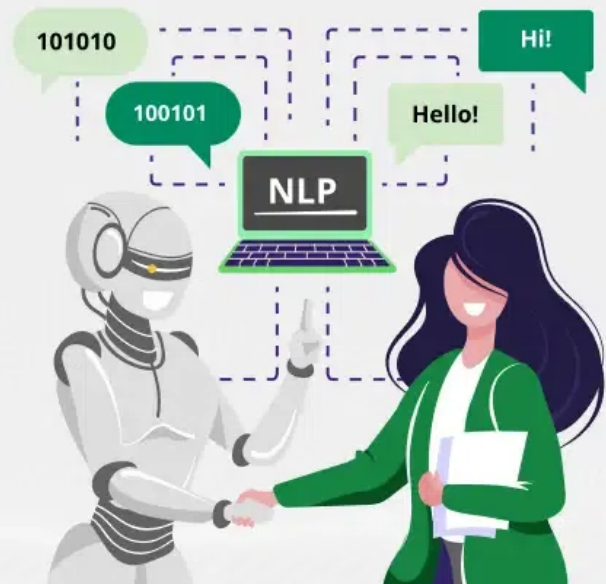


Introduction to Natural Language Processing (NLP)

Natural Language Processing (NLP) enables computers to understand and interpret human language. While computers excel at processing structured data, such as spreadsheets or databases, natural language in its unstructured form (text, speech, etc.) presents a unique challenge. NLP bridges this gap by allowing machines to process and understand human languages, making it an essential tool in modern AI systems.

Let's learn more about Natural Language Processing in this article.

Introduction to Natural Language Processing



Need for Natural Language Processing

The growing amount of unstructured natural language data in the world makes it increasingly important for machines to comprehend and analyze it effectively. By training NLP models, we can equip computers to process language data in a variety of forms, from written text to voice input. With centuries of human-written literature and massive data available, it's vital to teach computers to interpret that wealth of information. However, this task comes with significant challenges, such as resolving ambiguities in meaning, Named-Entity Recognition (NER), and coreference resolution.

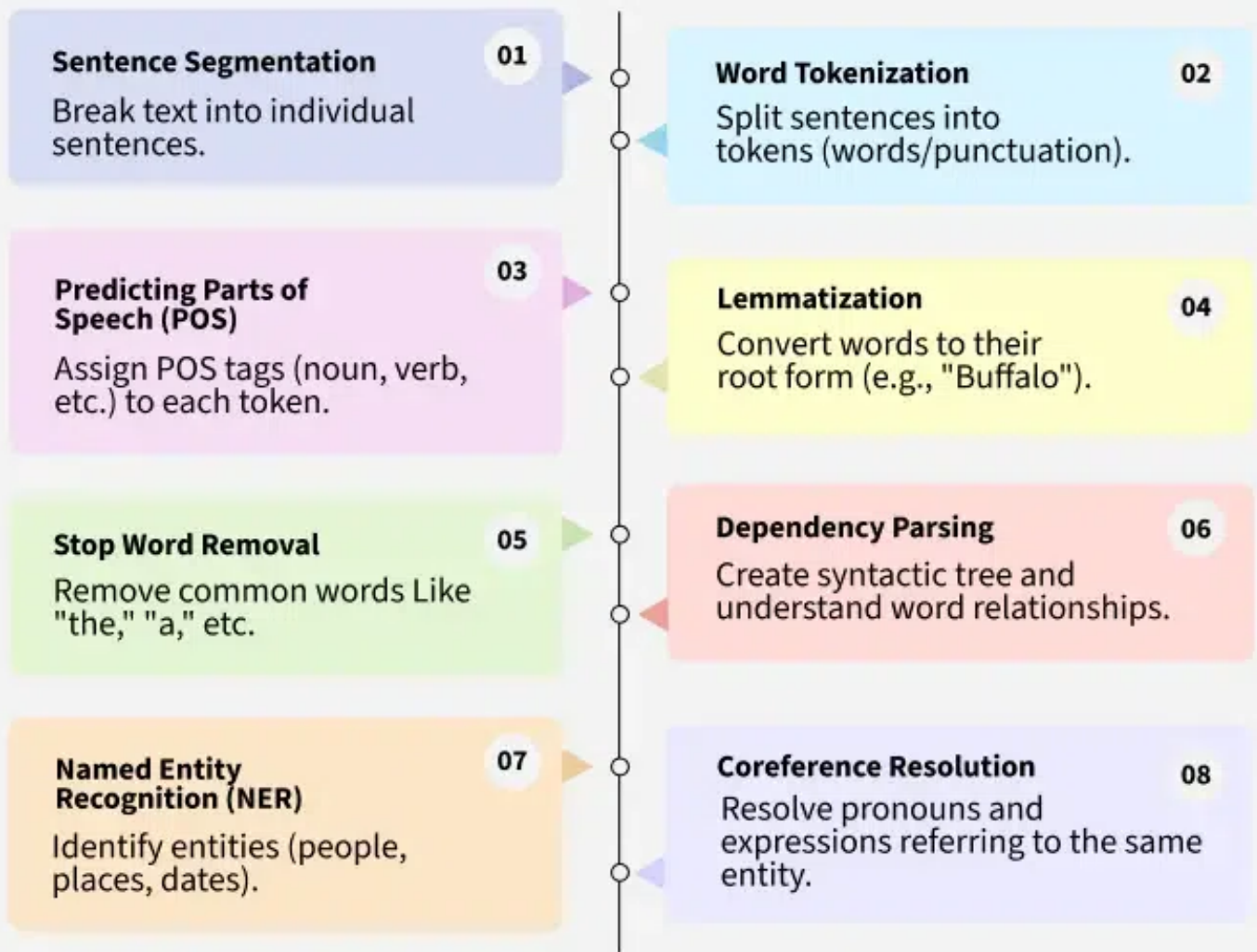
While NLP systems are improving, they still face difficulties in understanding the exact meaning of sentences.

For example, consider the phrase: "The boy radiated fire-like vibes." Does it refer to a motivating personality or imply something literal? Such ambiguities make text analysis complex for computers.

To solve these challenges, NLP breaks down language understanding into smaller, manageable components. This approach, known as the NLP pipeline, involves several stages that collectively enable machines to interpret human language effectively.

Key Steps in Natural Language Processing (NLP) Pipeline

Natural Language Processing Pipeline



1. Sentence Segmentation

Sentence segmentation is the first step in NLP, which involves breaking text into individual sentences. This helps the computer understand the structure of the text.

Example Input: San Pedro is a town on the southern part of Ambergris Caye in Belize. According to estimates, it has a population of 16,444.

Output:

- Sentence 1: San Pedro is a town on the southern part of Ambergris Caye in Belize.
- Sentence 2: According to estimates, it has a population of 16,444.

2. Word Tokenization

Word tokenization divides sentences into smaller components called tokens (words or punctuation). These tokens are crucial for understanding how a sentence is structured.

Example Input: San Pedro is a town in Belize.

Output: Tokens: ['San', 'Pedro', 'is', 'a', 'town', 'in', 'Belize']

3. Predicting Parts of Speech (POS)

POS tagging involves identifying the function of each word in a sentence, such as whether it's a noun, verb, or adjective. This helps determine the role each word plays in the context of the sentence.

Example Input: San Pedro is a town.

Output: 'San Pedro' - Noun, 'is' - Verb, 'a' - Article, 'town' - Noun

4. Lemmatization

Lemmatization converts words to their root forms. For example, "Buffalo" and "Buffaloes" are both lemmatized to "Buffalo," ensuring that variations of the same word are treated identically.

Example Input: There are Buffaloes grazing in the field.

Output: Buffalo (root word)

5. Stop Word Removal

Stop words (e.g., "a," "the," "and") are common words that provide minimal meaning. Removing them helps reduce noise and improve the efficiency of NLP models.

Prefix Tree-> /Users/devon/Developer/Machine_Learning_Algorithms/6_101/Week8/student_code/lab.py (autocomplete, autocorrect, not autosuggest)

6. Dependency Parsing

Dependency parsing identifies relationships between words, creating a syntactic tree. This helps understand the grammatical structure of a sentence and the roles of each word.

Example Input: San Pedro is an island in Belize.

Output: Parse Tree: 'San Pedro' (subject) → 'is' (verb) → 'island' (object)

Noun phrases group related words to represent a specific concept. In the sentence "The second-largest town in the Belize District," we can extract the noun phrase "second-largest town."

7. Named Entity Recognition (NER)

NER identifies and categorizes entities such as people, places, or dates in text.

Example Input: San Pedro is a town on the southern part of the island of Ambergris Caye in the Belize District of the nation of Belize, in Central America.

Output:

- San Pedro: Geographic Entity
- Ambergris Caye: Geographic Entity
- Belize: Geographic Entity
- Central America: Geographic Entity
-

8. Coreference Resolution

Coreference resolution identifies when two or more expressions in a text refer to the same entity. For example, the word "it" might refer to a specific person or thing earlier in the sentence.

Example Input: San Pedro is a town on the southern part of the island of Ambergris Caye. According to 2015 mid-year estimates, the town has a population of about 16,444. It is the second-largest town in the Belize District.

Output: "It" refers to "San Pedro."

Techniques Used in NLP

NLP techniques can be broadly categorized into two approaches:

1. Rule-based Methods: These involve manually created rules and heuristics to process language data. For example, defining patterns in language to extract meaning.
2. Machine Learning (ML) and Deep Learning (DL): These involve using algorithms to automatically learn from data and improve over time. ML models such as decision trees, support vector machines, and deep learning models like recurrent neural networks (RNNs) and transformers are commonly used in modern NLP. One of the most prominent breakthroughs in NLP in recent years has been the use of transformers, a type of deep learning architecture that powers models like BERT and GPT. These models excel at tasks like language understanding and generation, enabling applications like chatbots and automated content creation.

Applications of NLP

1. Speech Recognition: Converting spoken language into text, which powers voice assistants like Siri and Alexa.
2. Language Translation: Automatically translating text from one language to another (e.g., Google Translate).
3. Chatbots and Virtual Assistants: Understanding and responding to user queries in natural language (e.g., customer support).
4. Text Summarization: Condensing long documents into shorter summaries without losing important information.
5. Sentiment Analysis: Understanding opinions expressed in text, which is useful in social media monitoring, product reviews, and customer feedback analysis.
6. Information Retrieval: Enhancing search engines by interpreting and matching search queries with relevant documents.

Topics	Reading Reference
Regex	https://regexone.com
Bag of Words Model	https://machinelearningmastery.com/gentle-introduction-bag-words-model/
TF-IDF	https://stevenloria.com/tf-idf/ , http://blog.christianperone.com/2011/09/machine-learning-text-feature-extraction-tf-idf-part-i/ , http://aimotion.blogspot.com/2011/12/machine-learning-with-python-meeting-tf.html
Stemming and Lemmatization	https://nlp.stanford.edu/IR-book/html/htmledition/stemming-and-lemmatization-1.html
POS- Tagging	https://nlpforhackers.io/training-pos-tagger/ , https://pythonprogramming.net/part-of-speech-tagging-nltk-tutorial/ , https://towardsdatascience.com/part-of-speech-tagging-with-hidden-markov-chain-models-e9fcc835c0e ,
Parsing	https://www.commonlounge.com/discussion/1ef22e14f82444b090c8368f9444ba78
Named Entity Recognition	https://www.commonlounge.com/discussion/d5abf44e9d09490b8653dd75feff760e
Word Net	https://pythonprogramming.net/wordnet-nltk-tutorial/
Concept Net	http://conceptnet.io/ , http://tomkdickinson.co.uk/2017/05/combining-wordnet-and-conceptnet-in-neo4j/
Topic Modelling	https://www.youtube.com/watch?v=BuMu-bdoVrU
Word2Vec	https://arxiv.org/pdf/1411.2738.pdf , http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/ https://www.youtube.com/watch?v=TsEGsdVJjuA

POPULAR VIDEOS AND ARTICLES

Text mining (NLP) with Python – Implementation Tutorial	https://nbviewer.jupyter.org/github/TiesdeKok/Python_NLP_Tutorial/blob/master/NLP_Notebook.ipynb
NLP Workshop – Concrete Solution to Real World Problem	https://github.com/hundredblocks/concrete_NLP_tutorial
NLP With Deep Learning – Stanford School OF Engineering	https://www.youtube.com/watch?v=OQQ-W_63UgQ&list=PL3FW7Lu3i5Jsnh1rmUwq_TcyINr7EkRe6
Compilation of 150 NLP and Python Tutorial	https://medium.com/machine-learning-in-practice/over-150-of-the-best-machine-learning-nlp-and-python-tutorials-ive-found-ffce2939bd78
Step by Step Guide	https://blog.insightdatascience.com/how-to-solve-90-of-nlp-problems-a-step-by-step-guide-fda605278e4e
Ultimate Guide to Understand and Implement NLP	https://www.analyticsvidhya.com/blog/2017/01/ultimate-guide-to-understand-implement-natural-language-processing-codes-in-python/

Lexical Processing

Stemming

It is a **rule-based** technique that just chops off the suffix of a word to get its root form, which is called the 'stem'. For example, if you use a stemmer to stem the words of the string - "The driver is racing in his boss' car", the words 'driver' and 'racing' will be converted to their root form by just chopping of the suffixes 'er' and 'ing'. So, 'driver' will be converted to 'driv' and 'racing' will be converted to 'rac'.

You might think that the root forms (or stems) don't resemble the root words - 'drive' and 'race'. You don't have to worry about this because the stemmer will convert all the variants of 'drive' and 'racing' to those root forms only. So, it will convert 'drive', 'driving', etc. to 'driv', and 'race', 'racer', etc. to 'rac'. This gives us satisfactory results in most cases.

There are two popular stemmers:

- **Porter stemmer:** This was developed in 1980 and works only on English words. You can find all the detailed rules of this stemmer ++[here](#)++.
- **Snowball stemmer:** This is a more versatile stemmer that not only works on English words but also on words of other languages such as French, German, Italian, Finnish, Russian, and many more languages. You can learn more about this stemmer ++[here](#)++.

Lemmatization

This is a more sophisticated technique (and perhaps more 'intelligent') in the sense that it doesn't just chop off the suffix of a word. Instead, it takes an input word and searches for its base word by going recursively through all the variations of dictionary words. The base word in this case is called the **lemma**. Words such as 'feet', 'drove', 'arose', 'bought', etc. can't be reduced to their correct base form using a stemmer. But a lemmatizer can reduce them to their correct base form. The most popular lemmatizer is the **WordNet lemmatizer** created by a team of researchers at the Princeton university. You can read more about it ++[here](#)++.

Nevertheless, you may sometimes find yourself confused in whether to use a stemmer or a lemmatizer in your application. The following points might help you make the decision:

1. A stemmer is a rule based technique, and hence, it is much faster than the lemmatizer (which searches the dictionary to look for the lemma of a word). On the other hand, a stemmer typically gives less accurate results than a lemmatizer.
2. A lemmatizer is slower because of the dictionary lookup but gives better results than a stemmer. Now, as a side note, it is important to know that for a lemmatizer to perform accurately, you need to provide the **part-of-speech tag** of the input word (noun, verb, adjective etc.). You'll see learn POS tagging in the next session - but it would suffice to know that there are often cases when the POS tagger itself is quite inaccurate on your text, and that will worsen the performance of the lemmatiser as well. In short, you may want to consider a stemmer rather than a lemmatiser if you notice that POS tagging is inaccurate.

In general, you can try both and see if its worth using a lemmatizer over a stemmer. If a stemmer is giving you almost same results with increased efficiency than choose a stemmer, otherwise use a lemmatizer.

Tf-IDF

The bag of words representation, while effective, is a very naive way of representing text. It relies on just the word frequencies of the words of a document. But don't you think word representation shouldn't solely rely on the word frequency? There is another way to represent documents in a matrix format which represents a word in a smarter way. It's called the TF-IDF representation and it is the one that is often preferred by most data scientists.

The term TF stands for term frequency, and the term IDF stands for inverse document frequency. How is this different from bag-of-words representation? Professor Srinath explains the concept of TF-IDF below.

upGrad

TF-IDF MODEL

Document 1 : "Gangs of Wasseypur is a great movie. Wasseypur is a town in Bihar."

Document 2 : "The success of a song depends on the music."

Document 3 : "There is a new movie releasing this week. The movie is fun to watch."

	bihar	depends	fun	gangs	great	movie	music	new	releasing	song	success	town	wasseypur	watch	week
1															
2															
3															

$$tf_{t,d} = \frac{f_{t,d}}{\sum_t f_{t,d}}$$
$$tf_{movie,d1} = \frac{1}{7}$$



The TF-IDF representation, also called the **TF-IDF model**, takes into the account the importance of each word. In the bag-of-words model, each word is assumed to be equally important, which is of course not correct.

The formula to calculate TF-IDF weight of a term in a document is:

The log in the above formula is with base 10. Now, the tf-idf score for any term in a document is just the product of these two terms:

Higher weights are assigned to terms that are present frequently in a document and which are rare among all documents. On the other hand, a low score is assigned to terms which are common across all documents.

Now, attempt the following quiz. Questions 1-3 are based on the following set of documents: Document1: "Vapour, Bangalore has a really great terrace seating and an awesome view of the Bangalore skyline" Document2: "The beer at Vapour, Bangalore was amazing. My favourites are the wheat beer and the ale beer." Document3: "Vapour, Bangalore has the best view in Bangalore."

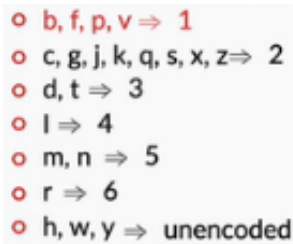
Phonetic hashing

is done using the Soundex algorithm. American Soundex is the most popular Soundex algorithm. It buckets British and American spellings of a word to a common code. It doesn't

matter which language the input word comes from - as long as the words sound similar, they will get the same hash code.

Now, let's arrive at the Soundex of the word 'Mississippi'. To calculate the hash code, you'll make changes to the same word, in-place, as follows:

1. Phonetic hashing is a four-letter code. The first letter of the code is the first letter of the input word. Hence it is retained as is. The first character of the phonetic hash is 'M'. Now, we need to make changes to the rest of the letters of the word.
2. Now, we need to map all the consonant letters (except the first letter). All the vowels are written as is and 'H's, 'Y's and 'W's remain unencoded (unencoded means they are removed from the word). After mapping the consonants, the code becomes MI22I22I11I.



- b, f, p, v ⇒ 1
- c, g, j, k, q, s, x, z ⇒ 2
- d, t ⇒ 3
- l ⇒ 4
- m, n ⇒ 5
- r ⇒ 6
- h, w, y ⇒ unencoded

- 3.
- 4.
5. The third step is to remove all the vowels. 'I' is the only vowel. After removing all the 'I's, we get the code M222211. Now, you would need to merge all the consecutive duplicate numbers into a single unique number. All the '2's are merged into a single '2'. Similarly, all the '1's are merged into a single '1'. The code that we get is M21.
6. The fourth step is to force the code to make it a four-letter code. You either need to pad it with zeroes in case it is less than four characters in length. Or you need to truncate it from the right side in case it is more than four characters in length. Since the code is less than four characters in length, you'll pad it with one '0' at the end. The final code is M210.

Phonetic correction

Our final technique for tolerant retrieval has to do with *phonetic* correction: misspellings that arise because the user types a query that sounds like the target term. Such algorithms are especially applicable to searches on the names of people. The main idea here is to generate, for each term, a ``phonetic hash'' so that similar-sounding terms hash to the same value. The idea owes its origins to work in international police departments from the early 20th century, seeking to match names for wanted criminals despite the names being spelled differently in different countries. It is mainly used to correct phonetic misspellings in proper nouns.

Algorithms for such phonetic hashing are commonly collectively known as *soundex* algorithms. However, there is an original soundex algorithm, with various variants, built on the following scheme:

1. Turn every term to be indexed into a 4-character reduced form. Build an inverted index from these reduced forms to the original terms; call this the soundex index.
2. Do the same with query terms.
3. When the query calls for a soundex match, search this soundex index.
The variations in different soundex algorithms have to do with the conversion of terms to 4-character forms. A commonly used conversion results in a 4-character code, with the first character being a letter of the alphabet and the other three being digits between 0 and 9.
4. Retain the first letter of the term.
5. Change all occurrences of the following letters to '0' (zero): 'A', 'E', 'I', 'O', 'U', 'H', 'W', 'Y'.
6. Change letters to digits as follows: B, F, P, V to 1. C, G, J, K, Q, S, X, Z to 2. D, T to 3. L to 4. M, N to 5. R to 6.
7. Repeatedly remove one out of each pair of consecutive identical digits.
8. Remove all zeros from the resulting string. Pad the resulting string with trailing zeros and return the first four positions, which will consist of a letter followed by three digits.
For an example of a soundex map, Hermann maps to H655. Given a query (say herman), we compute its soundex

code and then retrieve all vocabulary terms matching this soundex code from the soundex index, before running the resulting query on the standard inverted index.

This algorithm rests on a few observations: (1) vowels are viewed as interchangeable, in transcribing names; (2) consonants with similar sounds (e.g., D and T) are put in equivalence classes. This leads to related names often having the same soundex codes. While these rules work for many cases, especially European languages, such rules tend to be writing system dependent. For example, Chinese names can be written in Wade-Giles or Pinyin transcription. While soundex works for some of the differences in the two transcriptions, for instance mapping both Wade-Giles hs and Pinyin x to 2, it fails in other cases, for example Wade-Giles j and Pinyin r are mapped differently.

Syntactic Processing
